Stealth Intrusion Detection using TinyML Sound Classification on ESP32

PROJECT REPORT [Phase-I]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



Submitted by

Tanishq Yadav (24U020070 / 5th Semester)

Under the supervision of

Dr. Priyankar Choudhary

Assistant Professor

Computer Science & Engineering

November 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

CERTIFICATE

This is to certify that the work embodied in this report entitled **"Stealth Intrusion Detection using TinyML Sound Classification on ESP32"** has been satisfactorily completed by **Tanishq Yadav (24U020070).** It is a bona fide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering**, **Indian Institute of Information Technology, Bhopal,** for the partial fulfillment of the requirements for the Bachelor of Technology during the academic year 2026–27.

Date:

Dr. Priyankar Choudhary,
Assistant Professor,
Department of CSE,
IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia,
Department Coordinator,
Department of CSE,
IIIT Bhopal (M.P.)



INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

DECLARATION

I hereby declare that the following minor project report entitled "Stealth Intrusion Detection using TinyML Sound Classification on ESP32" is presented in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in** Computer Science and Engineering. It is an authentic documentation of my original work carried out under the able guidance of **Dr. Priyankar Choudhary**. The work has been carried out entirely at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole for the award of any degree or professional diploma in any other institute or organization.

I, with this, declare that the facts mentioned above are true to the best of my knowledge. In case of any unlikely discrepancy that may occur, I will be the one to take responsibility.

Tanishq Yadav (24U020070) Sign of student

ACKNOWLEDGEMENT

I, Tanishq Yadav, would like to express my sincere gratitude to everyone who supported and guided me throughout the successful completion of my minor project titled “Stealth Intrusion Detection using TinyML Sound Classification on ESP32”.

First and foremost, I would like to extend my heartfelt thanks to our esteemed **Director**, Dr. Ashutosh Kumar Singh, for providing me with the necessary resources and facilities for conducting my project. His encouragement and vision played a key role in helping me accomplish my goals.

I am deeply grateful to our **Project Guide**, Dr. Priyankar Choudhary, for his constant guidance, invaluable suggestions, and expert advice throughout the project. His knowledge and insights helped me enhance the quality of my work, and his support kept me motivated during difficult times.

My sincere thanks to the **Department Coordinator**, Dr. Rahul Chaurasia, for the valuable support and encouragement provided during the completion of the project.

I would also like to express my gratitude to all the **faculty** of the Department of Computer Science & Engineering for their valuable guidance and support in clarifying doubts and providing me with the right direction throughout the smooth execution of my project.

A special thanks to my **friends** for their continuous support, valuable suggestions, and helping me during various stages of the project. Their insights and encouragement motivated me to keep working on the project with enthusiasm.

Last but certainly not least, I would like to express my deepest gratitude to my **parents** and family members for their love, patience, and continuous moral support throughout the course of this project. Their understanding and encouragement were the driving force behind my efforts.

I am truly thankful to everyone who directly or indirectly contributed to the success of this project. Without their assistance and support, this project would not have been possible.

Tanishq Yadav

24U020070

AREA OF WORK

The idea for this project lies at the intersection of the three technologies – Embedded Systems, IoT, and Tiny Machine Learning. The central problem around which this project is formulated is a rather real one, and that is whether a trained model can be used on a resource-constrained microcontroller with minimal memory and computing power, and be effective. To answer that question, I put together audio signal processing, a neural network model, and cloud alerts, doing as much processing locally as possible.

When I began, however, the simple solutions seemed self-evident. I could either transmit the raw audio to the server and do classification there using a more complex model or use a conventional motion detector with a camera. However, both methods have significant drawbacks that could not be ignored. Server solution requires access to the Internet and often leaves Wi-Fi radio running continuously, thus wasting energy and becoming useless when the network disconnects. Moreover, audio transmitted from a closed room goes outside the building and it did not seem like a good idea to me. A motion sensor does not raise privacy concerns but is unable to distinguish between regular movements and the specific, quiet ones that this work aims at detecting. Therefore, I chose to do classification right on the ESP32 and wake other devices only in case of detection.

Classical machine learning algorithms have always been trained and executed on computers or servers. With the increasing proliferation of connected devices, this approach has turned out to be inadequate for decision-making tasks where rapid processing, low power consumption and privacy are important. The need to solve such problems led to the emergence of edge machine learning. Quantisation is an algorithm that allows reducing the size of the model and the amount of computation, and platforms like Edge Impulse significantly simplify building and executing a model on resource-constrained devices.

In this report I follow the whole processing pipeline. It starts with sampling audio from a MAX9814 microphone connected to the ESP32. The signal is converted into Mel-filterbank features and passed to a one-dimensional convolutional neural network (1D CNN) for classification. Because one wrong prediction should not wake the camera, I added a multi-window confirmation rule that has to be satisfied before the Wi-Fi radio and camera are switched on. After a confirmed event, the device takes a photograph, uploads it to the cloud and sends a push notification to a mobile app. Besides building this chain, I measure how well the trained model classifies and how much RAM, flash memory and inference time it needs on the target hardware.

TABLE OF CONTENT

S.no	Title	Page No.
	Certificate	i
	Declaration	ii
	Abstract	1
1	Introduction	2
2	Literature Review	5
3	Problem Definition and Objectives	12
4	Proposed Methodology and Work Description	14
5	Proposed Algorithms	22
6	Proposed Flowchart / Block Diagram / DFD	24
7	Implementation (Tools and Technology Used)	27
8	Result Discussion and Analysis	32
9	Conclusion and Future Scope	38
	References	40
	Plagiarism + AI Report (Turnitin)	42

LIST OF FIGURES

Fig	Description	Page no.
4.1	Signal path of the acoustic classifier, from raw samples to the deployed Arduino library	14
4.2	Data-recording setup: ESP32 with MAX9814 microphone connected to a laptop running Edge Impulse Studio	16
4.3	Mel-filterbank energy output of a stealth_movement training window (Edge Impulse Studio)	18
6.1	Block diagram of the proposed stealth intrusion detector	24
6.2	Flowchart of the detection and event-handling firmware	25
6.3	Data flow diagram of the system	26
6.4	Message sequence of the upload and notification path	26
8.1	Raw microphone signal in the serial plotter: (a) normal room, (b) intrusion	32
8.2	Results in Edge Impulse Studio: (a) validation set, quantised (int8) model; (b) test data, unoptimised (float32) model	33
8.3	Data explorer (correctly classified normal windows hidden) and on-device performance estimate in Edge Impulse Studio	35

LIST OF TABLES

Table No	Description	Page No.
2.1	Summary of related work	11
4.1	Hardware connections used in the prototype	15
4.2	Classes and recording plan of the acoustic dataset	17
4.3	Processing and learning blocks of the impulse	18
4.4	Optimisation steps applied before deployment	19
7.1	Hardware used in the prototype	27
7.2	Software tools and cloud services	27
7.3	Problems met during implementation and their solutions	31
8.1	Summary of the collected dataset	33
8.2	Confusion matrix on the test data (rows: true class, columns: predicted class)	34
8.3	Per-class results and overall metrics	34
8.4	Stealth scores from the serial log and the resulting hit counter	35
8.5	Resource usage on the ESP32-WROVER	36
8.6	Logged alert events during integration of the alert path	36
8.7	Comparison of the proposed system with other approaches	37

ABSTRACT

Most security systems still rely on motion sensors, such as passive infrared (PIR) sensors, or on video that is always on. If a person tiptoes in, both a burglar alarm and a motion detector detect this in much the same way as a person who is walking in normally. It is hard to separate the slow and quiet movement of a possible intruder from those of a non-intruder and the many false alarms caused by the cancel out the usefulness of such systems. I was curious to know whether sound alone can bridge that gap, and for that purpose I built a sonic intrusion detection system that listens to the environment and looks for this kind of movement. The prototype uses an ESP32-WROVER module with a MAX9814 analog microphone.

A small TinyML model is run on the microcontroller and makes real-time classification. Besides, there is no transmission of speech to the cloud. To avoid false alarms and save energy, an intrusion will be considered valid when three consecutive classifications pass the threshold. In case of validity, the system enters a cooldown mode for 60 seconds. A snapshot of the event is uploaded to Cloudinary for record purposes, and a push notification is made to a mobile application so that the user may see it.

Acoustic sensing is applied in the first level only. In the next level, I plan to apply the use of seismic sensors so that vibration sensing would complement the acoustic sensing. Based on my data, the model proved to have achieved 93.8% of validation accuracy and 90.09% of test accuracy and classified 95.5% of stealth movement windows. From the live testing, the prototype has proved to go through all the alert process, uploading the image in the cloud and giving a notification in the phone.

Github_repository: https://github.com/Tanishq4009/stealth-detection

INTRODUCTION

1.1 Background

Security and intrusion detection systems are used in homes, offices, laboratories, classrooms and storage rooms. Most of them are based on passive infrared (PIR) sensors, door and window contacts, or CCTV cameras that continuously record everything [1], [2]. These techniques are effective at detecting movement or the opening of a door. They fail when they have to distinguish normal movement from deliberately quiet movement [2], [3]. This project started from something very ordinary: a person arriving late to class and trying to make his way in without a sound. A motion detector would treat that person the same way as anyone else, regardless of how he moved. This made me wonder whether a system could identify an unusual silence from sound and record evidence only when a possible intrusion was actually confirmed. The classroom was just the beginning. Since the same problem exists in homes, offices and warehouses, I created the project for indoor intrusion detection in general and not for one kind of room.

1.2 Limitations of existing systems

Here are some flaws of existing systems and how they have influenced the design of my system [1]–[3]:

- PIR sensor limitations: A PIR sensor responds to the infrared radiation of warm moving bodies in front of it [1]. It responds to both humans and animals. It has no mechanism to interpret the movement pattern of a person, which restricts its ability to identify suspicious behaviour and leads to false alarms [2], [3].
- Too many resources for surveillance: Most CCTV systems run in 24x7 mode. Hence, they consume power, storage and network bandwidth around the clock, and hours of footage have to be reviewed to find one specific event of interest. Motion-triggered capture, as used in ESP32-based systems [2], [4], is one way of avoiding continuous recording.
- Privacy concerns: Constant recording is a threat to the privacy of the place, as it records people even when nothing interesting is happening. Processing the data on the device, as TinyML allows [5], [6], is one way of limiting this exposure.
- Cost and complexity of the system: Some commercial smart security systems are costly to install and maintain. Some depends heavily on cloud services and internet connectivity and they become mostly useless without it. Low-cost ESP32-based systems have been proposed as an affordable alternative [2], [4].

In general, what is needed is a system that stays mostly in a low-power idle state during normal situations and triggers only when there is an event of interest. This would reduce unnecessary recording, save power and improve privacy [2], [5].

1.3 The concept of “stealth” intrusion

Before a system can detect a careful intruder, I had to decide what “stealth” actually means in terms that can be measured [7].

Conventional acoustic sensors, such as glass-break detectors, are made for loud or sudden sounds. They raise an alert when the sound level crosses a fixed threshold, which works well for a sharp noise but not for someone who is deliberately trying to stay quiet [1], [8].

Stealthy movement means soft steps, slow motion and avoiding anything sudden. The sound it produces can be so faint that it is hard to separate from ordinary background noise, so checking whether the volume exceeds a threshold is not enough [9], [10].

The system has to look at subtle acoustic cues and at how the sound pattern changes over time. A machine learning model can learn these characteristics, and that is what lets it recognise stealthy movement without waiting for a loud noise [9], [11]. This idea is the basis of the acoustic approach used in this project.

1.4 Paradigm shift: edge computing and TinyML

My interest in edge computing and Tiny Machine Learning (TinyML) was piqued by the challenge of detecting patterns in sound at a low-power level while keeping the audio on the device. TinyML enables a machine learning model to operate on a small, low-power microcontroller rather than on a powerful computer or a cloud server. This allows a device with little memory and processing power to analyse data and decide locally [5], [6]. The audio in this project is handled by the ESP32 and is not streamed as raw audio to a server. Audio is held for short durations, analysed by the neural network and discarded. The on-device strategy avoids all three drawback of cloud processing: latency, consumption of Wi-Fi bandwidth and privacy risk [12]. To reduce reliance on the network and lower power use, the Wi-Fi connection and the camera are turned on only after an event has been verified.

1.5 Project motivation and objectives

The project focuses on developing a smart sound-based security system that overcomes the various disadvantages of traditional systems [2], [3]. I established three primary goals.

- Low cost: I used less expensive hardware, the ESP32-WROVER microcontroller and the MAX9814 microphone, to keep the price low, with future enhancements possible through modifications to the design [2], [13].
- Low-power operation: The system keeps listening to sound all the time but processes very little at any moment. By using sound analysis instead of continuous video to detect a suspected intrusion, much of the unnecessary power consumption of traditional systems can be avoided [2], [5].
- Event-active camera: The camera does not have constant surveillance, but it works on an event-based system. It begins with analysing the audio signals to detect any stealth entry, and only after detection do the camera and Wi-Fi begin operating. It takes one

photograph and uploads it online, thus having low usage of the camera and only visualising major events [2], [4].

1.6 Organisation of the report

The rest of this report is arranged as follows. Chapter 2: Literature Review discusses earlier work on intrusion-detection sensing [1]–[3], [14], footstep acoustic detection [7], [9], [10], environmental sound classification [11], [15], [16], seismic sensing [8], sensor fusion [3], [14], TinyML on microcontrollers [5], [6], [17]–[19] and ESP32-based security systems [2], [4]. This project addresses the gap identified in that study.

Chapter 3 lays out the problem [2], [3], the objectives, the scope, the constraints and assumptions, and the success criteria.

Chapter 4, Proposed Methodology and Work Description, gives details about hardware and signal conditioning, data collection and labelling, feature extraction and the 1D CNN [19], [20], model optimisation and deployment, confirmation logic, and event handling with Cloudinary and OneSignal [21], [22].

The algorithms for the sampling task [23], detection with confirmation, and the event routine are proposed in Chapter 5.

The system block diagram, the firmware flowchart, the data flow diagram and the sequence of the alert path [21], [22] are presented in Chapter 6: Proposed Flowchart / Block Diagram / DFD.

Chapter 7 covers the hardware [13], [24], [25], the software tools and cloud services [17], [21], [22], the firmware routines, the mobile application and the problems I ran into during implementation.

Discussion and Analysis of Results Chapter 8 covers explanations about the dataset, classifier performance analysis, the influence of the confirmation rule, the study of resource usage [26], analysis of upload and notification, comparison of the proposed system to other methods [2], [3], [9], [10] and the system's limitation.

Chapter 9 summarises the outcomes and improvements, including the addition of seismic sensors for multimodal intrusion detection.

LITERATURE REVIEW

2.1 Overview

This chapter looks at the research behind a low-cost security device that detects quiet, deliberate movement through sound [2], [9] and takes a photograph only after a possible intrusion is confirmed.

These six themes were covered in the literature review: intrusion detection technologies [1], [2], footsteps and movement detection using acoustics [7], [9], [10], classification of environmental sounds, vibration/seismic sensors, sensor fusion, and machine learning based security systems using ESP32-WROVER [2], [4]. In all of these areas, the contribution of the existing research, their findings and their shortcomings have been reviewed. The chapter concludes with a comparison of the reviewed research and the gap in the research addressed by this study (Section 2.9).

The papers were obtained from academic databases and search engines like arXiv, IEEE Xplore, MDPI, and Google Scholar, based on search terms like footstep sound classification [9], [10], acoustic intrusion detection [14], seismic footsteps detection [3], TinyML audio classification [18], [19], and ESP32-WROVER surveillance [2], [4]. Each paper was analysed in regard to its objectives, sensor technology, approach used, findings and limitations. It is important to note that the limitations are what matter most since they explain the need for the proposed system.

2.2 Intrusion detection sensing technologies

PIR sensors are common in low-cost security systems because they detect movement without a camera running all the time [1], [2].

The principle of operation of a PIR sensor lies in detecting changes in infrared emissions from objects that are moving within its range of vision [1]. Okokpujie et al. [2] developed an ESP32-WROVER surveillance system where a PIR sensor activates the camera, the images are recorded to a microSD memory card, while SMS notifications are sent via the Twilio application programming interface. Furthermore, a frame averaging method for motion detection based on difference analysis of images' frames exceeding the threshold value is mentioned. Although the authors claim that the motion detector can be used both indoors and outdoors, they do not provide any statistics on detection efficiency, false alarms, and false negatives, which are listed among their future research directions. The stated range of detection is around 20 feet.

False alarms are more of an issue for PIR in general. Jin et al. [3] examined unattended ground sensors that used a combination of seismic and PIR sensors and found out that false positives were a reason why they did not work well, and that was due to limitations in their algorithm processing. Huang et al. [14] implemented seismic, acoustic, and ultrasonic sensors at the border checkpoints to distinguish between animals and humans.

There are two main conclusions drawn from these experiments that are important to my work. Firstly, a device that can only sense the motion of a warm object has nothing to say about the way in which the motion happens, which is a drawback when it comes to detecting quiet motion. Secondly, the previous research on decreasing false alarms was based on using multiple modalities and classifying algorithms [3], [14].

Acoustic and vibration sensing look like a useful complement. They can detect certain events in darkness and, depending on the signal and the sensor position, without a direct line of sight [7], [9]. That makes them good candidates for deciding when a security camera should be triggered, namely only when the activity gives enough evidence of a possible intrusion.

2.3 Detecting footsteps and human movement with sound

Footstep acoustics have been studied for detecting people and for recognising individuals. These studies show that walking sounds carry useful information about a person's presence, movement pattern and identity [7], [9], [10].

Qu et al. [9] combined acoustic signal processing with deep learning to detect and recognise several people from indoor footstep sounds. Their dataset was based on a simulated family of three, and the work concentrated on family recognition and activity detection. In Qu's doctoral thesis [7], footstep sound features were used to classify different walking styles. The thesis examined how Doppler shifts relate to footstep vibrations, built a deep learning system for detecting several people who walk at the same time, and pointed out how hard it is to keep real-time performance and accuracy when many people walk together.

Huang and Wang [10] looked at pedestrian detection from walking sounds, aiming at devices such as voice assistants. Their approach combines a convolutional neural network (CNN), signal processing designed for footstep sounds and stereo audio recording to detect footsteps in complex indoor environments, and they report an identification accuracy of up to 94.9% from a single footstep. In a separate live-footstep evaluation they used features within and between footsteps, together with spatial variation, to defend against replay attacks. Their method reportedly blocked all conventional replay attacks and more than 99% of advanced attacks that imitate both the acoustic and spatial characteristics of footsteps. This result matters to my project because it exposes a weakness of sound-based detection: a recording of footsteps can be played back to imitate real movement.

The following three facts are significant for the research. Firstly, the reviewed papers focus on recognition of humans rather than stealth intrusion, which is characterised by silent and careful movements [7], [9], [10]. Secondly, the research requires stereo audio recording or use of even more advanced microphone array [10], while my algorithm uses only one cheap MAX9814 microphone that is connected to the ESP32-WROVER device. Thirdly, it is unclear from the paper descriptions whether these machine learning classification models are able to be executed on a microcontroller with limited resources.

2.4 Classifying environmental sounds and audio features

Environmental Sound Classification (ESC) is the closest relative of the classification stage in my system, since both identify sound events from audio recordings [15], [27].

Nogueira et al. [15] reviewed audio recognition for smart-city applications, including security and surveillance. They note that urban sound classification is hard because real recordings mix unstructured, overlapping and unrelated sounds. The review points to deep learning architectures, attention mechanisms, data augmentation and pre-training as the main techniques for better performance. The best results came from a pre-trained DenseNet-161 with two data augmentation techniques, reaching 97.98% on UrbanSound8K, 98.52% on ESC-50 and 99.22% on ESC-10. The authors also stress that how these models behave in real-world environments has not been studied enough, so it is hard to say how well benchmark numbers carry over to practice.

Feature selection is another key point. Huzaifah [11] compared several feature extraction methods, namely short-time Fourier transforms (STFTs) on linear and Mel scales, the constant-Q transform and the continuous wavelet transform, with Mel-frequency cepstral coefficients (MFCCs) as the baseline. The features were evaluated with convolutional neural networks on ESC-50 and UrbanSound8K. The Mel-scaled STFT did slightly better than the other spectral methods and clearly better than MFCCs. The study also found that the best window size depends on the audio, and that two-dimensional convolution generally beat one-dimensional convolution. Zhang et al. [16] used mixup, a data augmentation method that blends training samples and their labels, with a deep convolutional neural network. Their best settings gave 83.7% on UrbanSound8K, 83.9% on ESC-50 and 91.7% on ESC-10, and they also studied how mixup affects accuracy and feature distributions.

Two things follow for my system. First, the findings of Huzaifah [11] encouraged me to try Mel-based features, so I used Mel-filterbank energy (MFE) features, which describe the frequency content of the audio well and are still practical to compute on an embedded system. Second, the data augmentation work of Nogueira et al. [15] and Zhang et al. [16] is relevant because the dataset I collected is small. I did not use augmentation in the current model (Table 4.3), so it remains future work.

At the same time, the values for accuracy are based on public data and do not represent the model's performance in a specific instance. According to Nogueira et al. [15], performance is an issue still to be investigated since a model which was trained on benchmark data may demonstrate different performance in a specific environment with a particular microphone used.

This is why my work was based mostly on the data obtained using my own microphone setup [15]. The reason for choosing MFE features is explained in Section 4.5 while the accuracy of the model is reported in Section 8.3.

2.5 Seismic and vibration-based detection

Footfalls also generate vibrations that can travel via floors or the ground surface and these can be detected using geophones and accelerometers [3], [14]. The use of seismic sensors has been

explored extensively for detecting personnel, particularly in the context of unattended ground sensor networks for perimeter monitoring applications.

Huang et al. [14] have used seismic sensors to detect improved summary autocorrelation patterns and have used temporal data collected by ultrasonic sensors to distinguish human motion from the motion of four-legged animals. Jin et al. [3] have fused seismic and PIR sensor data and have performed wavelet transforms and probabilistic finite state automata to detect low dimensional features. It has been claimed that the method does not require significant computation and memory resources.

The link between footstep sounds and ground vibrations has been explored in acoustic research as well. Qu's doctoral thesis [7] looked at the connection between Doppler shifts in footstep sounds and vibration characteristics, and used it to improve the recognition of walking styles and postures.

Seismic sensing has been used in intrusion detection for decades. An early patent describes a system with a geophone, a band-pass amplifier working in the 10–40 Hz range and a threshold detector that counts seismic events caused by people or vehicles [8]. So ground vibration has long been a trigger for possible intrusions. The catch is that such systems usually need buried sensors or sensors mechanically coupled to the ground. The current phase of my project deals with airborne sound, so I did not need any ground-coupled sensor.

The seismic studies I reviewed [3], [14] address unattended ground sensing and border or perimeter surveillance, and both aim to separate human activity from other sources and cut false alarms. Neither looks at combining airborne sound and floor vibration to identify quiet, deliberate movement indoors. I think this is worth exploring in a later phase, where seismic sensing could support the existing acoustic detection.

2.6 Multimodal sensor fusion

Sensor fusion is widely used to reduce false alarm in detection system, as noted in Section 2.2 [3], [14].

Huang et al. [14] tried two ways of combining acoustic, seismic and ultrasonic data. One was decision-level fusion with Gaussian mixture model classifiers, the other was feature-level combination with support vector machines. The authors report that both improved reliability, but the abstract I could access gives no quantitative results, dataset details or sample sizes, so I could not judge the performance independently.

Jin et al. [3] combined seismic and PIR data with a cross-probabilistic finite-state automaton, which models the relationship between the two signals and produces low-dimensional feature vectors. They tested it on seismic and PIR datasets for target detection and classification, but the abstract does not give accuracy figures or false alarm rates.

Both studies suggest that combining sensing methods improve reliability, because one sensor can make up for the weakness of another [3], [14]. Their focus, however, is unattended ground sensors and border or perimeter surveillance. In the abstracts I reviewed, I did not find a comparable low-cost indoor system where sensor-based detection decides when a camera is activated.

The fusion I plan for a later phase has a different purpose. It is not just about confirming that a person is present [3], [14]. It asks whether the relationship between airborne sound and floor vibration can say something about how the person is moving. For example, a footstep that gives a weak acoustic signal but a detectable floor vibration could indicate quiet movement. This is my own design idea and not an established finding from the studies above, so it will have to be tested in experiments in a future phase.

2.7 TinyML and machine learning on microcontrollers

Microcontrollers have little memory and processing power, so a machine learning model has to be compact and efficient to run on them. Saha et al. [5] review machine learning on microcontroller-class hardware and describe the steps and practical considerations in developing and deploying such models, while also pointing out the problems that are still unsolved. Garai and Samui [6] concentrate on lightweight keyword-spotting models, an audio task common in always-listening devices. They explain that deploying these models on edge devices needs efficient TinyML techniques, and they sort the approaches into seven categories: model design, learning methods, model compression, attention-based architectures, feature optimisation, neural architecture search and hybrid approaches.

Edge Impulse supports much of this workflow. Hymel et al. [17] describe it as a cloud-based machine learning platform for embedded and edge devices that supports a wide range of hardware, and say that it helps with the compatibility problems between hardware and software environments that make optimisation and deployment difficult. By October 2022, 50,953 developers had created 118,185 projects on the platform.

Several studies have used this workflow for audio and movement tasks on development boards. Viswanatha et al. [18] used the Arduino Nano 33 BLE Sense, which has a 32-bit Arm Cortex-M4F processor running at 64 MHz, for accelerometer-based hand gesture recognition and for keyword spotting that triggers an LED. They used Edge Impulse for spectral-analysis preprocessing, a neural network classifier built with Keras and an 80/20 training-test split, and exported the model as an Arduino library. The paper shows working implementations but does not report accuracy values for the final models.

The speech-recognition system in [19] uses a one-dimensional convolutional neural network (1D CNN) to recognise 23 keywords on the same board, with TensorFlow Lite and Edge Impulse. The authors report an average accuracy of 0.97 and an average F1-score of 0.98. They also list limitations: the dataset has recordings from one speaker only, the board's built-in microphone is of limited quality, the noise classes generated automatically by Edge Impulse are not clearly defined, and the system was tested on a single hardware platform.

These studies gave me a foundation for my approach: collect data, extract audio features and train a lightweight neural network that runs directly on a microcontroller [17]–[19]. The examples, however, are mostly about speech and gestures [18], [19], not about movement patterns such as walking or stealthy footsteps.

Some limitations reported in [19] showed up in my own implementation as well. A low-quality built-in microphone, or data collected under limited conditions, can hurt how well a model

works in its real environment. The handling of automatically generated noise classes also taught me how important it is to control the training data. My first model had only two classes, *stealth* and *normal*. It sometimes predicted stealth with high confidence even when there was no meaningful sound, because it had no separate class for background noise and nowhere else to put such input. The final model still has only these two classes, so the limitation has not been removed. I reduced its effect with the confirmation rule (Section 4.7), and a separate background-noise class is planned as future work (Section 9.2).

2.8 ESP32-WROVER-based security and surveillance systems

The ESP32-WROVER packs a microcontroller, Wi-Fi and camera support onto one small, cheap board, so it turns up often in security projects [2], [4], [13].

Two recent papers show the usual designs. Okokpujie et al. [2] use an ESP32-WROVER with an OV2640 camera and a PIR sensor. The PIR output is read on GPIO 12, pictures go to a microSD card, SMS alerts are sent through the Twilio API, and video can be viewed remotely in an app. Incidentally, GPIO12 is the strapping pin that stopped my own board from booting when I wired the microphone to it (Section 4.2). They tested indoors and outdoors and say the running cost is lower than that of existing systems, though they do not give the exact cost. Their limitations are the lack of measured detection accuracy and false alarm figures, a PIR range of about 20 feet, and notification delays that depend on network traffic.

Humam and Triawan [4] built a room-security system in which an ESP32-WROVER takes pictures when a motion sensor detects movement, and the sensor data and images go to a cloud server for processing and storage.

A mobile app sends alerts to the user [4]. The authors say the system is fast and sends notifications within seconds, but the abstract has no detailed test data and does not name the sensor.

Both systems follow the same pattern: a motion sensor decides when to capture images, and the cloud or a phone handles the rest [2], [4].

A motion trigger only tells you that something moved. In the two systems I looked at there is no classifier on the device and no use of sound to detect movement. My project uses the same kind of low-cost board and cloud upload as [2] and [4], but replaces the PIR sensor with an audio classifier running on the device.

The ESP32-WROVER also has hardware limits that the paper abstracts do not mention, such as limited memory when the camera and Wi-Fi run together [13], [24]. I ran into this myself. To upload over HTTPS I had to switch off the camera first, and I had to keep Wi-Fi disabled except while uploading (see Section 4.8).

2.9 Summary of related work and research gap

Table 2.1 summarises the papers discussed above [2]–[4], [7], [9], [10], [14], [18], [19]. The entries are taken from the abstracts, and “not stated” means the abstract did not say.

Table 2.1: Summary of related work

Ref.	Sensor(s)	Method	Platform	Stated or observed limitation
[9]	Not stated	Acoustic signal processing with deep learning, multi-person footstep recognition	Not stated	Simulated three-person setting; no numerical metrics in abstract
[7]	Not stated	Footstep-sound posture classification; Doppler-vibration correlation; deep learning	Not stated	Real-time and multi-person accuracy remain challenges
[10]	Stereo microphones	CNN footstep identification with replay-attack detection	Voice-assistant devices	Identifies people; does not target quiet movement
[14]	Seismic, acoustic, ultrasonic	Exemplar-based acoustic classifier; GMM decision fusion; SVM feature fusion	Not stated	No quantitative results in abstract; border-crossing use
[3]	Seismic, PIR	Wavelet and PFSA feature extraction; cross-PFSA fusion	Unattended ground sensors	No accuracy or false-alarm figures in abstract
[2]	PIR, OV2640 camera	PIR trigger; frame-difference motion check; SMS alerts	ESP32-CAM	No accuracy or false-alarm rates; PIR range about 20 ft
[4]	Motion sensor (model not stated)	Capture on motion; cloud storage; app notification	ESP32-CAM	No detailed test data in abstract
[19]	On-board microphone	1D CNN keyword recognition; Edge Impulse	Arduino Nano 33 BLE Sense	One speaker; poor microphone; opaque noise classes
[18]	IMU, microphone	Edge Impulse spectral analysis and Keras network	Arduino Nano 33 BLE Sense	No numerical accuracy reported
This work	MAX9814 microphone	Edge Impulse audio classifier; confirmation logic; trigger-based photo upload	ESP32-WROVER	Phase-I audio only; small own dataset

Three gaps stand out in the papers I examined. First, the studies on footsteps and sounds aim to locate, count or recognise people [7], [9], [10], and none of them checks whether someone is moving quietly on purpose. Second, work that combines audio with ground vibration concentrates on specialised sensors and border surveillance [3], [14], and leaves out a cheaper system for indoor use. Third, the ESP32-WROVER security systems start recording from a conventional PIR motion detector [2], [4], while the audio studies that use machine learning aim at speech and gestures [18], [19], not at walking behaviour.

This project brings these three gaps together: detection of quiet movement by sound, a low-cost microcontroller that processes the data on board, and a camera that switches on only when a real event occurs [2], [9], [19].

PROBLEM DEFINITION AND OBJECTIVES

3.1 Problem statement

Most conventional security systems can detect movement, but they cannot tell how a person is moving.Suppose someone enters a room slowly and quietly to avoid making noise. A motion sensor, or a camera triggered by movement, will react almost as it would for someone walking in normally. These systems can also raise false alarms because of harmless activity [2], [3]. Continuous recording catches every event, but it uses a lot of power and storage and keeps recording when nothing suspicious is happening.

This project tries to deal with these limitations by building a low-cost, low-power security device with the following aims:

- Develop a device that detects quiet, deliberate movement by analysing the surrounding sound.
- Verify a possible event before acting on it, then capture and send a single photograph, keeping the camera and wireless module off when they are not needed.
- Deal with the difficulty of detecting stealthy movements, which make very little sound and can easily hide into background noise.
- Build a lightweight sound classifier that separates relevant sound pattern from background noises and runs directly on a microcontroller, instead of relying on a fixed volume threshold.

The main challenge is to pick out meaningful movement from subtle sound patterns while keeping the system efficient enough for low-power, on-device operation.

3.2 Objectives

1. Create a labelled dataset to train the model by recording:
    - Sounds produced by stealthy movements.
    - Samples of normal background sounds.

2. Use the same microphone and hardware setup for most of the recordings to ensure that the model is trained under the same conditions in which the device will operate.

3. Detect different sounds with Edge Impulse on the ESP32-WROVER.
    - Use Edge Impulse to analyse the sounds and extract useful features from the recordings.
    - Train a model that can identify various audio events.
    - Run the model locally on the ESP32-WROVER to classify sounds in real time without requiring a network connection.

5. Reduce false positives.
    - Add functionality to the ESP32 to accept detections only when certain conditions are met, such as a high confidence score, multiple detections in quick succession, and/or a predefined waiting time.

6. In case intruders are identified, perform the following actions:
    - Turn the camera on.

- Capture one image.
- Upload the image to the cloud and turn Wi-Fi off once more.

7. Test the system for performance under realistic conditions.
8. Test the system on the test data set which differs from the training data set.
9. Report the limitations of the system and their ways of improvement.

3.3 Scope

This report covers the first phase of development, which uses a single microphone for audio-based sensing and is tested indoors.

The following are planned for later phases and are outside the scope of the present work:

- Adding a vibration sensor to study the relationship between sound and ground vibration.
- Developing an automatic calibration mechanism so that the system can adapt to different rooms and acoustic conditions.

3.4 Constraints and assumptions

Hardware limits

The ESP32-WROVER has limited memory, so the camera and the secure upload cannot run at the same time. Some GPIO pins are also unreliable or unusable: GPIO12 is a strapping pin that is read at start-up, and some ADC2 pins do not work while Wi-Fi is on.

Single signal

A sound-only system can be fooled by someone who knows how it works, and it can miss movement that makes almost no noise. This is one reason why later phases add another sensor.

3.5 Success criteria

1. Model Accuracy:

- Validation accuracy: target of at least 90%; achieved 93.8%.
- Test accuracy: target of at least 90%; achieved 90.09% (refer to Section 8.3 and the confusion matrix).

2. Detection Performance:

- Few false triggers during idle monitoring; 6 were observed in 30 minutes (refer to Section 8.4).
- Correct detections in the stealth-movement test; 19 were observed (refer to Section 8.4).

3. Image Upload Performance:

- Image upload success rate and latency (refer to Section 8.6).

4. Deployed Model Characteristics:

- RAM consumption: small enough for the ESP32-WROVER; measured 10.8 kB.
- Flash storage: measured 40.0 kB.
- Inference time: fast enough to run on every audio window; approximately 1 ms (refer to Section 8.5).

PROPOSED METHODOLOGY AND WORK DESCRIPTION

4.1 Overview of the proposed methodology

The system listens continuously to the sound in a room through a single microphone, and all audio classification happens on the device. The camera and Wi-Fi are switched on only when the system picks up a possible stealth movement event. After the event is confirmed, it captures one photograph, uploads it to a cloud image service and sends a push notification to the owner's phone. While nothing is detected, the camera and Wi-Fi stay off. I divided the work into seven stage, each covered in this chapter: (1) hardware assembly and signals conditioning, (2) acoustic data collection, (3) dataset preparation and labelling, (4) features extraction and model training, (5) model optimisation and deployment on microcontroller, (6) real-time inference with an event confirmation rule, and (7) event handling, which covers photo capture, image upload and notification.

The most important design decision was to classify on the device instead of on a remote server. Sending raw audio to a server would keep the Wi-Fi connection on all the time, which increases power use and makes the system depend on the network. It would also send audio recorded inside a private space out of the building. With a compressed model running on the microcontroller, the audio is processed locally, and only the detection result and one photograph leave the device when an event is confirmed. The device can also keep detecting movement when the internet connection is down [12]. Figure 4.1 shows the signal flow from the microphone to the deployed classifier.



Figure 4.1: Signal path of the acoustic classifier, from raw samples to the deployed Arduino library

4.2 Hardware platform and signal conditioning

The main controller is an ESP32-WROVER module, which has a dual-core processor, built-in Wi-Fi and external PSRAM [13]. The PSRAM is especially useful here because it holds the JPEG image frame and the multipart upload buffer, which leaves the internal RAM free for the TLS connection. An OV2640 camera is connected using the AI-Thinker board's pin configuration, which the open-source camera driver for this module supports [24]. For audio I used a MAX9814 amplifier breakout with an electret microphone, a low-noise preamplifier and an automatic gain control (AGC) stage, whose gain can be set to 40, 50 or 60 dB through the GAIN pin [25].

Getting a usable audio signal depended on the GAIN pin. At first I left it unconnected, which gives the amplifier its 60 dB gain, and the ADC readings sat at 4095 even in a quiet room.

Connecting the GAIN pin to the supply voltage dropped the gain to 40 dB, and the readings started to move around the mid-scale value as the sound changed. The AR (attack/release) pin is left unconnected, so the amplifier works with its default AGC timing.

The amplifier output goes to GPIO14, which belongs to the ESP32's ADC2 block. ADC2 cannot be used while Wi-Fi is active, and this single limitation decides how the firmware has to work: audio is sampled only when Wi-Fi is off, and the wireless connection is turned on only after sampling has stopped. Table 4.1 lists all the hardware connections.

Table 4.1: Hardware connections used in the prototype

Signal	From	To	Remark
Microphone supply	MAX9814 VDD	ESP32 3.3 V	Common ground with the module
Microphone output	MAX9814 OUT	GPIO14 (ADC2)	12-bit reading, range 0-4095
Gain select	MAX9814 GAIN	3.3 V	40 dB, avoids saturation
Attack/release	MAX9814 AR	Not connected	Default AGC timing
Camera data and control	OV2640 D0-D7, XCLK, PCLK, VSYNC, HREF, SIOD, SIOC	GPIO 5, 18, 19, 21, 36, 39, 34, 35; 0, 22, 25, 23; 26, 27	AI-Thinker pin map
Camera power-down	OV2640 PWDN	GPIO32	HIGH = camera off
Programming	FTDI TX/RX	ESP32 U0R/U0T	GPIO0 to GND only while flashing

Debugging this stage taught me three hardware lessons that shaped the final design. First, GPIO12 is a strapping pin that sets the flash voltage at start-up. When I connected the microphone output to it, the board would not boot and printed the error “invalid header: 0xffffffff”, so I moved the microphone output to GPIO14. Second, GPIO32 is already used for the camera's power-down function, so it cannot be used for the microphone. Third, the camera and the Wi-Fi module can draw a large current when they run together, and this caused resets until I gave the system a separate 5 V supply (Table 7.3).

4.3 Acoustic data acquisition

The audio is read with the on-chip ADC at 8 kHz. I chose this rate because the sounds I care about, such as footsteps, cloth friction and floor creaks, carry most of their energy at low and middle frequencies, and 8 kHz keeps the data volume and processing time small enough for a microcontroller. The MAX9814 output sits on a DC offset of about half the supply voltage. Before feature extraction I remove this offset by estimating the DC level with a moving average (low-pass) and subtracting it from every sample, so that the features describe the sound and not the bias of the circuit.

Sampling runs on a dedicated FreeRTOS task pinned to core 0, so that inference on core 1 cannot disturb the timing of the samples [23]. The task reads one sample every 125 microseconds and writes it into one half of a double buffer. When that half is full, the two halves are swapped and a flag signals that a new slice of audio is ready. The classifier therefore always works on one buffer while the other is filling, and no sample is lost.

4.4 Dataset creation and labelling

A model trained on public sound datasets may not work as well with a different microphone or in a different room, because the frequency response, gain and background noise of the recording setup all influence what the model learns. Sound event detection in real environments has been studied widely [27], but datasets such as ESC-50 [28] and the Speech Commands corpus [29] were recorded with other equipment and for other purposes. To bring my training data closer to the final system, I recorded most of the audio myself with the same MAX9814 microphone, gain setting and ESP32 pin configuration as the deployed device. The audio goes to Edge Impulse Studio through the Data Forwarder as raw ADC values, which matches the input the device sees in operation [30].

Figure 4.2 shows the recording setup. The ESP32 board with the microphone attached lies on the floor and is connected to a laptop by USB. Edge Impulse Studio runs on the laptop, and I select the class label before each recording.

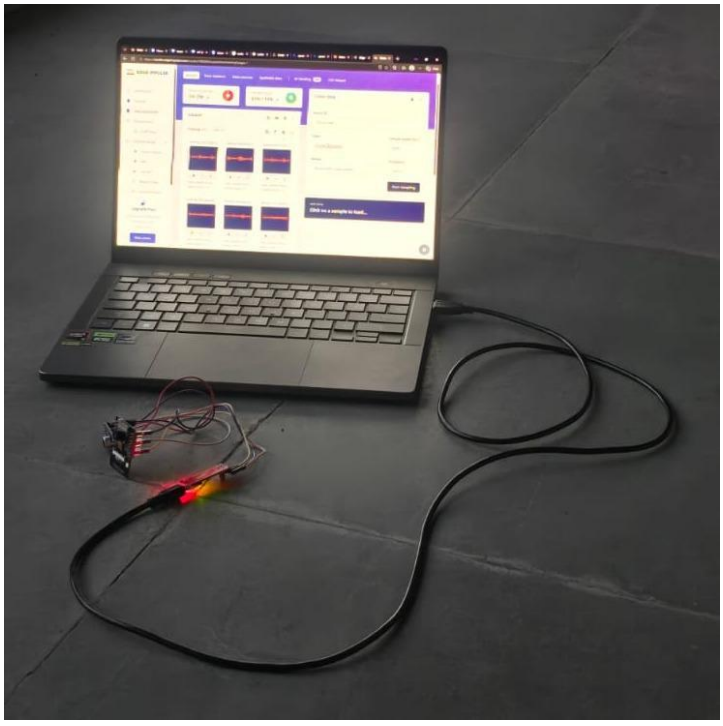


Figure 4.2: Data-recording setup: ESP32 with MAX9814 microphone connected to a laptop running Edge Impulse Studio

Some of the recordings were not made by me and were obtained from free online sources. The ceiling-fan recording was obtained from YouTube [31], the tiptoe recording from the Freesound website [32] and all other recordings from Pixabay [33]. These recordings were made with other microphones (i.e., with frequency responses different from that of the

MAX9814) and were uploaded as training inputs through the data acquisition page of Edge Impulse Studio. They are included to add variety to the background recordings. A breakdown of recordings from each source is given in Section 8.2.

The dataset has two classes: stealth_movement and normal_movement. The former includes quiet walking, sliding feet and gentle movements of doors or cloth. The latter includes normal walking, talking and background noise. Using only two class keep the model and the labelling simple, but quiet background noise that sound like stealth movement can cause false alarm. To reduce the risk of false alarms, I ignore uncertain prediction and ask for several consecutive high-confidence detections (see the confirmation rule in Section 4.7). In future work, I plan to add a third class for background noise. Table 4.2 outlines these classes and the recording plan.

Table 4.2: Classes and recording plan of the acoustic dataset

Class	What is recorded	Variation planned
stealth_movement	Slow, careful walking and sneaking; sliding steps; quiet door and cloth sounds	Several people, footwear types, floor types and distances from the microphone
normal_movement	Ordinary walking, talking, moving objects, normal room sound	Same people and places as above

I split the recordings into equal-length windows and kept separate sessions for training and testing. This stops similar windows from the same recording ending up in both sets, which would inflate the accuracy. The final dataset size is reported in Chapter 8.

4.5 Feature extraction and model design

The 500 ms time window of the audio signal is converted into a log-scaled Mel-filterbank energy spectrogram (MFE). This retains all the spectral characteristics of the signal, and the various frequency bands are mapped to a Mel scale, where the frequency bands are distributed in relation to how humans perceive the signal. It thereby eliminates any need for a further decorrelation step in the case of MFCC features [20]. Time-frequency representation selection remains an active area of research in the context of environmental sounds. There have been several studies that compare common techniques and indicate the efficacy of Mel based features for environmental sound classification [11], [34].

An example of short audio from my training samples of "stealth_movement" – maybe a footfall or a move of clothes - is clearly distinguishable from the background noise in Figure 4.3. I have used MFE instead of MFCC for my primary feature extraction as I wanted to detect sounds and not speech. MFCC requires some compression, which can erase the fine nuances. MFE retains the spectral details required to extract the nuances of soft footfalls. Both MFCC and MFE are supported by Edge Impulse; I might want to try MFCC at some point.

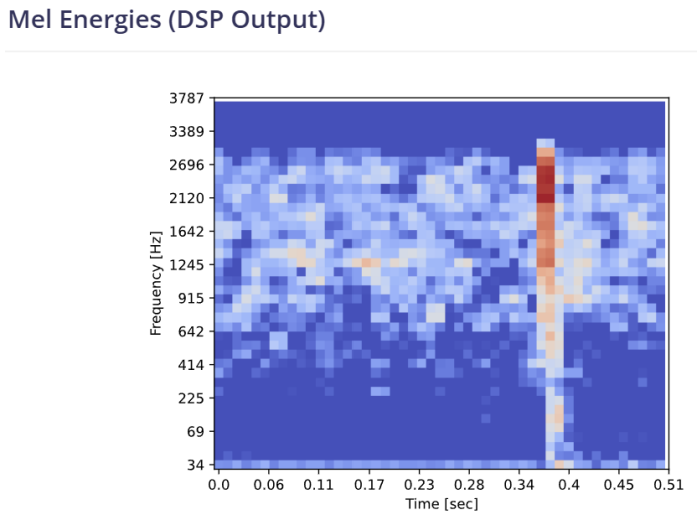


Figure 4.3: Mel-filterbank energy output of a stealth_movement training window (Edge Impulse Studio)

The classifier is a lightweight one-dimensional convolutional neural network (1D CNN) that processes the MFE features of each audio window, reshaped into 40 columns. It has a convolutional and pooling layer with 8 filters and kernel size 3, followed by a dropout layer with rate 0.25. A second convolutional and pooling layer with 16 filters and the same kernel size comes next, then another dropout layer with rate 0.25 and a flatten layer. The final layer is a softmax layer with two outputs, one for each class.

The softmax layer gives two values between 0 and 1 that add up to 1, and the one for stealth_movement is used as the detection confidence in the later stages. I kept the network small on purpose. It has to run on every audio window while sampling continues, and a bigger network could use so much processing time and memory that nothing would be left for the camera and the network. Table 4.3 summarises the processing and learning blocks.

Table 4.3: Processing and learning blocks of the impulse

Block	Setting
Input	Raw ADC samples, 8 kHz, window size as set in the impulse design (window size: 500 ms)
Pre-processing	DC offset removal by moving-average (low-pass) estimate
Feature block	Mel-filterbank energy (MFE): frame length 0.02 s, frame stride 0.01 s, 40 filters, FFT length 256, low frequency 0 Hz, high frequency left at the default, noise floor -52 dB
Learning block	1D convolutional neural network with softmax output, 2 classes: reshape (40 columns), Conv1D 8 filters (kernel 3), dropout 0.25, Conv1D 16 filters (kernel 3), dropout 0.25, flatten
Training	200 epochs, learning rate 0.005, validation split 20%, and no data augmentation.

4.6 Model optimisation and deployment

After training, I quantised the model from 32-bit floating-point values to 8-bit integers (int8). This cuts its size to roughly a quarter and allows faster integer arithmetic, usually with only a small loss in accuracy [35]. The quantised model is compiled with the Edge Impulse EON compiler, which generates C++ code instead of relying on an interpreted model graph and so needs less RAM and flash than a general-purpose interpreter such as TensorFlow Lite for Microcontrollers [12], [30]. The compiled model is exported as an Arduino C++ library and built into the project sketch together with the rest of the firmware. Table 4.4 lists these steps with the measurements reported by Edge Impulse Studio, which I discuss in Section 8.5.

Table 4.4: Optimisation steps applied before deployment

Step	Purpose	Measured effect
int8 quantisation	Smaller model, integer arithmetic	Int8 validation accuracy 93.8% (deployed model)
EON compiler	Lower RAM and flash than an interpreter	Peak RAM 10.8 kB, flash 40.0 kB (Studio estimate)
Arduino library export	Direct inclusion in the sketch, offline operation	Inference time about 1 ms (Studio estimate)

4.7 Real-time inference and confirmation logic

The classifier runs in continuous mode, meaning that the prediction window slides forward in short hops in time and predictions are made on each new segment of audio that enters the window. This way, the classifier can recognise events immediately without waiting for a longer audio recording, which gives faster detection of audio events. The classifier runs in the main loop on core 1, while the separate sampling task on core 0 fills the audio buffer.

One high-confidence prediction does not sound a warning; there must be a series of factors to confirm an event. Firstly, the value of stealth_movement should attain the level of confidence, which is 0.70. Secondly, this must be true for 3 consecutive prediction, so that an instant increase in noise would not generate a false alarms. Thirdly, at least 60 second must pass since the last upload.

The reason for the first two criteria is to eliminate false positives. For example, if someone slams a door open suddenly, the sound may be classified with high confidence, but it is most likely due to a one-time event. Requiring 3 consecutive decision windows helps to filter such events out. The 60-second cooldown mean the system does not send a large number of photos and notifications while someone keeps walking around the room, although some users might want exactly that.

The detection counter is also updated on a cycle-by-cycle basis, that is, once per complete model window. Getting three hits in a row therefore mean three whole windows in which the system judged that the sound resembled stealthy movement. Once a window falls below the threshold, the count resets to zero. All these parameters are near the top of the firmware, which makes them easy to adjust when testing in environments with different acoustics.

4.8 Event handling: photo capture, upload and push notification

When the alarm condition is met, the sampling task stops and the system runs a fixed sequence that fits within the memory and wireless limits of the microcontroller.

1. **Connect to Wi-Fi.** The Wi-Fi radio is switched on and the device tries to join the access point. If the connection still fails after a limited number of retries, the event is discarded and the radio is switched off again, so the detector cannot get stuck at this step.
2. **Check internet connectivity.** The device does a DNS lookup for the image service to confirm that internet access is available.
3. **Capture an image.** The camera is powered on and initialised in JPEG mode at VGA resolution, with a single frame buffer in PSRAM. A few initial frames are captured and thrown away so that the sensor's automatic exposure can settle, and the next frame is kept for upload.
4. **Prepare the upload buffer.** The captured frame is copied into a multipart/form-data buffer in PSRAM. The camera driver is then de-initialised before the upload, which frees the internal RAM it was using and leaves more memory for the TLS handshake. In early development the upload failed with a negative error code whenever I skipped this step.
5. **Upload the photograph.** The image buffer is sent to Cloudinary's image API in an HTTPS POST request. I used an unsigned upload preset, so no secret key has to be stored for the image upload [21].
6. **Send a notification.** After a successful image upload (HTTP 200), the device sends another HTTPS POST request to OneSignal's REST API. The request contains the application identifier, target segment, notification title and message, and the address of the uploaded image in an additional-data field called image_url [22]. OneSignal delivers the notification to the phone through the platform's push service. The mobile application, built with Flutter and the OneSignal SDK, shows the alert and reads image_url from the notification to display the photograph stored on Cloudinary.
7. **Resume monitoring.** The Wi-Fi radio is switched off, the audio buffers and model state are reset, and the sampling task starts again. The cooldown timer starts at this point too.

I used a push notification service because connecting the microcontroller directly to the platform's push servers is less practical. The newer HTTP v1 interface of Firebase Cloud Messaging needs short-lived OAuth 2.0 tokens signed with a private key, and generating those on an ESP32 would put extra load on its memory and power. The OneSignal REST API needs only an HTTPS request with a static key, and the service takes care of the tokens, so the embedded side stays fairly simple. The REST API key is still a secret and has to be protected, as discussed in Section 7.6.

4.9 Testing and evaluation plan

The evaluation follows the criteria in Section 3.5. First, I test the classifier offline on a test set taken from sessions that were not used for training, and report a confusion matrix with precision, recall and F1 score for each class. After deployment, I test the complete device in three ways: (a) an idle test, where the device runs in a quiet room for a fixed time and I count the false triggers; (b) a detection test, where volunteers walk quietly and normally through the room and I count the correct detections; and (c) an end-to-end test of the time from the trigger to the photograph in the cloud and the notification on the phone. Resource use (RAM, flash, inference time) is read from the compiler report and the serial monitor. All results are given in Chapter 8, and any value that has not been measured yet is marked as such and not estimated.

PROPOSED ALGORITHMS

5.1 Algorithm overview

Three routines perform different roles within the firmware. The first, a sampling routine running as a task on core 0, handles audio sampling. The second, a detection routine running in the main loop on core 1, applies the classifier and checks the confirmation condition. The third runs once an alarm has been confirmed and performs what I call the event routine.

5.2 Algorithm 1: sampling task (core 0)

```
Audio sampling task
Input : sampling rate Fs = 8000 Hz, buffer length N
Output: double buffer B[0], B[1] filled with raw ADC samples
 1: select <- 0; count <- 0; Ts <- 1/Fs
 2: while recording is enabled do
 3:     t0 <- now()
 4:     B[select][count] <- ADC_read(GPIO14)
 5:     count <- count + 1
 6:     if count = N then
 7:         select <- 1 - select; count <- 0; mark buffer ready
 8:     end if
 9:     wait until now() - t0 >= Ts
10: end while
```

5.3 Algorithm 2: detection with confirmation (core 1)

```
Classification and confirmation rule
Input : trained model M, THRESH, HITS_REQUIRED, COOLDOWN
 1: hits <- 0; last_upload <- 0
 2: loop
 3:     wait for a ready buffer; x <- DC_remove(buffer)
 4:     f <- MFE(x)                    { time-frequency features }
 5:     p <- softmax(M(f))             { p_stealth, p_normal }
 6:     if p_stealth > THRESH then hits <- hits + 1 else hits <- 0
 7:     if hits >= HITS_REQUIRED and (now() - last_upload > COOLDOWN
 8:                                or last_upload = 0) then
 9:         hits <- 0
10:         stop sampling task
11:         EventRoutine()
12:         last_upload <- now()
13:         reset buffers and model state; restart sampling task
14:     end if
15: end loop
```

5.4 Algorithm 3: event routine (capture, upload, notify)

```
EventRoutine
 1: Wi-Fi ON; wait for connection (at most R retries)
 2: if not connected or DNS lookup fails then Wi-Fi OFF; return
 3: camera ON; init(JPEG, VGA, 1 frame buffer in PSRAM)
 4: discard k warm-up frames; frame <- capture()
 5: body <- multipart(preset, frame) in PSRAM
 6: release frame; camera deinit; camera OFF
 7: code <- HTTPS_POST(Cloudinary, body)
```

```
8: if code = 200 then HTTPS_POST(OneSignal, alert + image URL)
9: free body; Wi-Fi OFF
```

5.5 Complexity and resource considerations

The sampling routine does a fixed amount of work for each sample, and the detection routine has a constant cost per slice, because the feature image and network sizes are fixed at design time. The processing time therefore stays predictable during continuous operation.

The event routine takes much more time and power, which is why it runs only after all three confirmation conditions are met. Algorithm 3 manages its memory in two ways: large buffers go in PSRAM, and the camera driver is de-initialised before the TLS connection is opened.

PROPOSED FLOWCHART / BLOCK DIAGRAM / DFD

6.1 System block diagram

Figure 6.1 illustrates the elements of the system architecture. There are two paths that process data for different purposes. The first is an always-on processing chain made up of the microphone, the sampling task and the classifier, followed by the confirmation logic, which passes data on to the event-handling part once an event is confirmed.

The second path contains the event-handling components: the camera, image upload, push notification and mobile application. The confirmation logic is the most important element of the whole structure, since it keeps all event-handling components silent until the system itself has confirmed an intrusion.

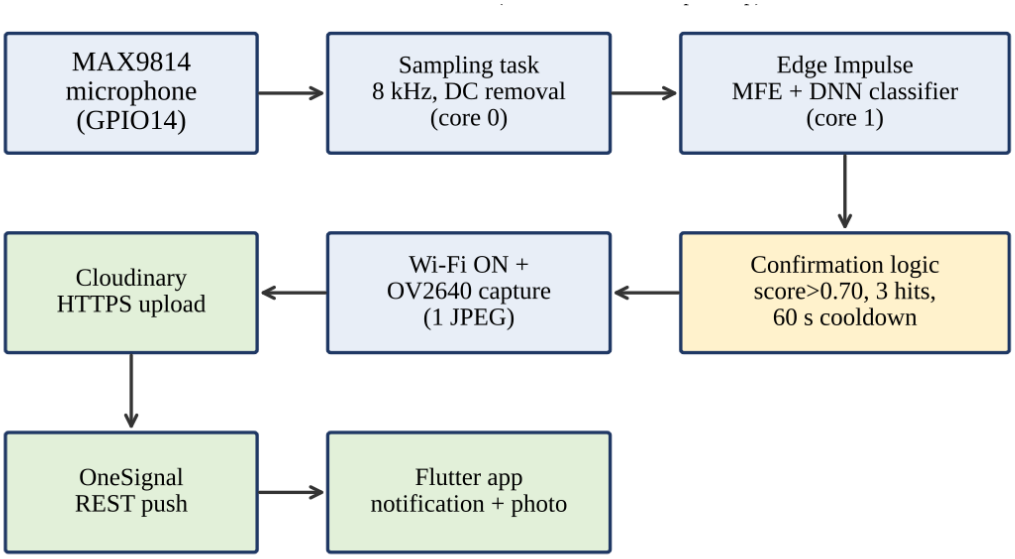


Figure 6.1: Block diagram of the proposed stealth intrusion detector

6.2 Flowchart of the firmware

Figure 6.2 shows the workflow, which follows the same logic as Algorithms 1–3. After start-up the Wi-Fi radio and camera stay off, the model is loaded and audio sampling begins. Every time a complete audio window is available, its features are extracted and passed to the classifier.

If the predicted probability is below the threshold, the detection counter is reset and the system keeps listening. If it is above the threshold, the counter goes up. The device leaves the listening loop only after three consecutive detections have been recorded and the cooldown has expired. It then runs the image capture, upload and notification sequence, and when that is done it goes back to listening.

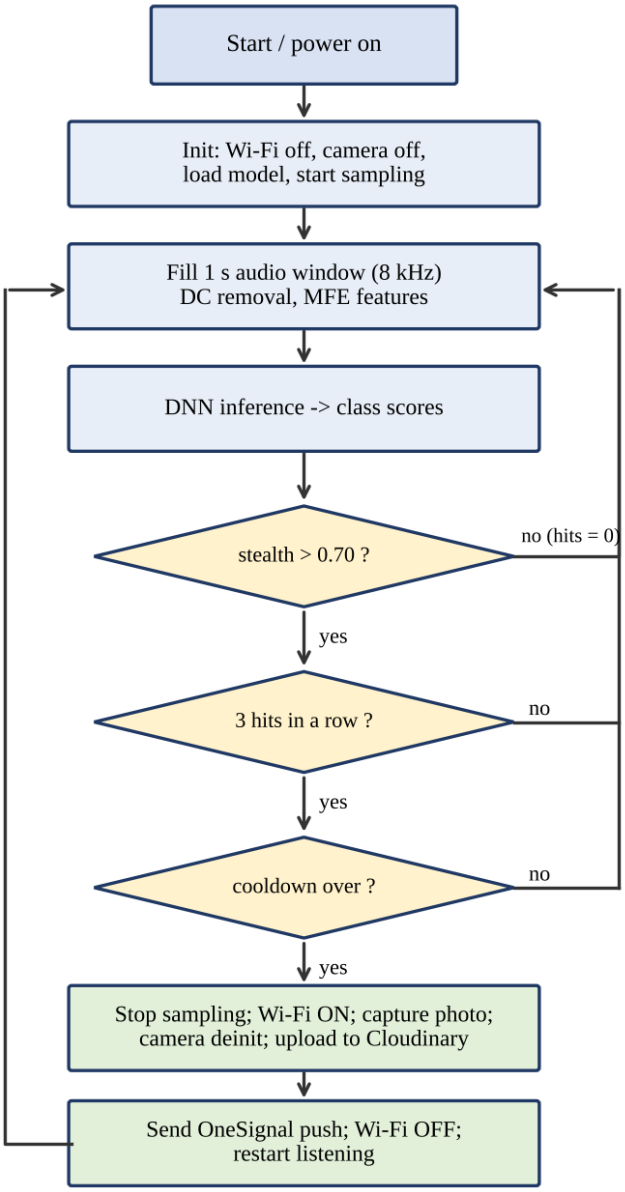


Figure 6.2: Flowchart of the detection and event-handling firmware

6.3 Data flow diagram

As shown in Figure 6.3, environmental sound reaches the first stage of processing (Process 1.0) and is then split into audio windows. Process 2.0 takes over with feature extraction. Using this information along with the stored quantised model, Process 3.0 identifies the sounds the system was trained on and confirms whether an event is likely to be associated with them at that time.

Once an event has been confirmed, Process 4.0 captures a photograph of the surroundings. This photo is passed to Process 5.0, which uploads the image to the cloud service in use.

Only the photo (and the accompanying message) leaves the device; all other information remains on the microcontroller and is never uploaded to the cloud.

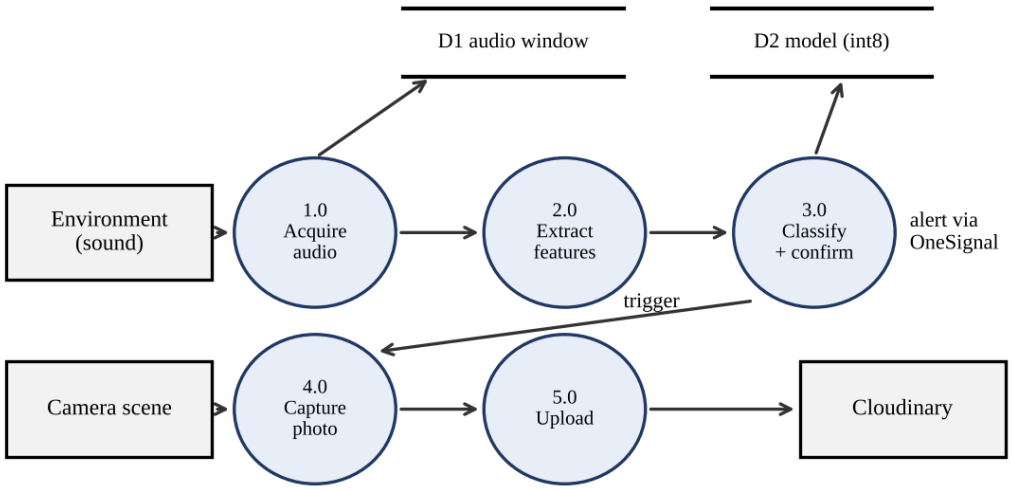


Figure 6.3: Data flow diagram of the system

6.4 Sequence of the alert path

This section describes the flow that takes place once an event has been confirmed. Figure 6.4 shows how the components handle the event. After the capture, the ESP32 uploads the picture to the cloud platform (Cloudinary). As soon as the image has been uploaded, Cloudinary returns an HTTP 200 response along with its secure URL (secure_url). As illustrated in the figure, the ESP32 then sends an alert message through OneSignal, and the notification is delivered to the Flutter app running on the user's smartphone. The URL used for displaying the picture (from Cloudinary) is in the notification's image_url parameter. The Flutter app simply reads it and displays the image hosted on Cloudinary.

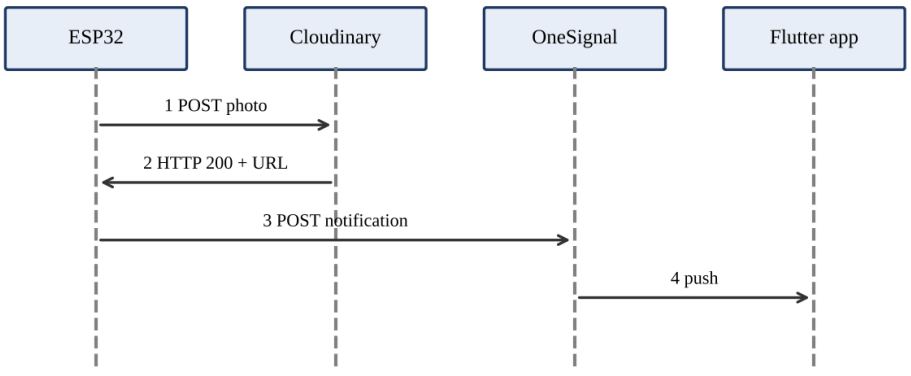


Figure 6.4: Message sequence of the upload and notification path

IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

7.1 Hardware tools

Table 7.1 lists the hardware components used in the prototype and the role of each component in the system [13], [24], [25].

Table 7.1: Hardware used in the prototype

Component	Specification	Role in the project
ESP32-WROVER module	Dual-core 240 MHz, Wi-Fi, PSRAM	Sampling, inference, networking, camera control
OV2640 camera	2 MP, JPEG output, AI-Thinker pin map	One photograph per confirmed event
MAX9814 microphone amplifier	Electret microphone, AGC, gain 40/50/60 dB	Audio sensor (gain pin set to 40 dB)
USB-to-serial adapter (FT232RL)	3.3 V logic level	Programming and serial monitor
Separate 5 V supply	5 V, enough current for camera and Wi-Fi peaks	Stable power for the whole system
Smartphone	Runs the Flutter application	Receives the push notification

7.2 Software tools and services

Table 7.2 lists the software tools and cloud services used for firmware development, model training, image storage and notification delivery [17], [21], [22].

Table 7.2: Software tools and cloud services

Tool / service	Use
Arduino IDE with the ESP32 board package	Firmware development, board selection, upload
Edge Impulse Studio	Data forwarding, labelling, MFE, training, int8 quantisation, EON compilation, Arduino library export
FreeRTOS (inside the ESP32 core)	Task on core 0 for sampling
esp32-camera driver	OV2640 initialisation and capture
WiFiClientSecure and HTTPClient libraries	HTTPS requests to Cloudinary and OneSignal
Cloudinary (unsigned upload preset)	Cloud storage of the photographs
OneSignal REST API and SDK	Push notification delivery
Flutter with onesignal_flutter and cached_network_image	Mobile application that receives the alert and shows the photograph

7.3 Development environment and board settings

I developed the firmware in C++ with the Arduino IDE. The board is set up as an ESP32 module with PSRAM enabled and a large-application partition scheme. This was needed

because the model library, camera driver and TLS stack together need more space than the default application partition has.

The camera pins follow the AI-Thinker pin mapping in Table 4.1. To flash the firmware, GPIO0 has to be held low while the board is reset, and the jumper must be removed once the upload is done. Otherwise the board stays in bootloader mode and never starts the application.

7.4 Implementation of the detection loop

The detection loop runs the Edge Impulse continuous classifier whenever a new audio slice arrives. It reads the probability of the stealth_movement class, updates the detection counter and checks the three conditions from Section 4.7. The listing below shows the decision part of the loop. Sensitive values such as the Wi-Fi network name and API keys are replaced with placeholders.

```
const float        STEALTH_THRESHOLD = 0.70;   // minimum confidence
const uint8_t      HITS_REQUIRED     = 3;      // consecutive windows
const unsigned long COOLDOWN_MS      = 60000;  // gap between uploads

// inside loop(), after run_classifier_continuous(...)
if (stealth_score > STEALTH_THRESHOLD) stealth_hits++;
else                                   stealth_hits = 0;

bool cooldown_over = (last_upload_ms == 0) ||
                     (millis() - last_upload_ms > COOLDOWN_MS);

if (stealth_hits >= HITS_REQUIRED && cooldown_over) {
  stealth_hits = 0;
  record_status = false;          // stop the sampling task
  delay(100);
  trigger_camera_and_upload();    // Wi-Fi, camera, upload, notify
  last_upload_ms = millis();
  restart_listening();            // clean buffers, restart task
}
```

7.5 Implementation of sampling and restart

The sampling task runs on core 0 with a 4096-byte stack and high priority. It reads values from the ADC, fills the double buffer and uses a short busy-wait to hold the sampling interval at 125 microseconds. After an event has been handled, the task is deleted. The restart_listening() function then resets the buffer indices, reinitialises the continuous classifier to clear any old state, and creates the sampling task again.

```
static void capture_samples(void* arg) {
  const uint32_t interval_us = 1000000 / EI_CLASSIFIER_FREQUENCY;
  while (record_status) {
    uint32_t t0 = esp_timer_get_time();
    int16_t sample = analogRead(MIC_PIN);      // GPIO14
    inference.buffers[inference.buf_select][inference.buf_count++] = sample;
    if (inference.buf_count >= inference.n_samples) {
      inference.buf_select ^= 1;
      inference.buf_count = 0;
      inference.buf_ready = 1;
      vTaskDelay(pdMS_TO_TICKS(1));
    }
```

```
    while (esp_timer_get_time() - t0 < interval_us) { }  // keep Fs stable
  }
  vTaskDelete(NULL);
}
```

7.6 Implementation of the upload and notification routine

The trigger_camera_and_upload() function implements Algorithm 3. The listing below shows its main steps in simplified form. The Cloudinary request sits in its own code block, so that its TLS connection is closed and its memory released before the next request starts.

Once Cloudinary replies, the function pulls the photograph's URL out of the response and puts it in the notification sent to OneSignal. The notification goes out only when Cloudinary returns HTTP 200, so the owner is never alerted about an image that was not stored.

```
// 1-2. Wi-Fi on, connectivity check
WiFi.mode(WIFI_STA);
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);          // placeholders
...                                            // up to 20 retries, 500 ms each
if (!WiFi.hostByName("api.cloudinary.com", ip)) { /* abort, Wi-Fi off */ }

// 3. Camera on, VGA JPEG, one frame buffer in PSRAM
digitalWrite(PWDN_GPIO_NUM, LOW);
config.frame_size = FRAMESIZE_VGA;  config.jpeg_quality = 12;
config.fb_count = 1;  config.fb_location = CAMERA_FB_IN_PSRAM;
esp_camera_init(&config);
for (int i = 0; i < 3; i++) { esp_camera_fb_return(esp_camera_fb_get()); delay(100); }
camera_fb_t *fb = esp_camera_fb_get();

// 4. Copy into one multipart body in PSRAM, then free the camera
requestBody = (uint8_t*)ps_malloc(prefixLen + fb->len + suffixLen);
memcpy(...);                                   // prefix + JPEG + suffix
esp_camera_fb_return(fb);
esp_camera_deinit();
digitalWrite(PWDN_GPIO_NUM, HIGH);

// 5. HTTPS upload to Cloudinary (own scope: TLS memory freed afterwards)
String imageUrl = "";  bool uploaded = false;
{
  WiFiClientSecure client;  client.setInsecure();   // prototype only
  HTTPClient http;
  http.begin(client, "https://api.cloudinary.com/v1_1/" + cloudName + "/image/upload");
  http.addHeader("Content-Type", "multipart/form-data; boundary=" + boundary);
  int code = http.POST(requestBody, totalLen);
  if (code == 200) {
    String resp = http.getString();            // JSON reply of Cloudinary
    int s = resp.indexOf("\"secure_url\":\"") + 14;
    imageUrl = resp.substring(s, resp.indexOf("", s));
    uploaded = true;
  }
  http.end();
}
free(requestBody);

// 6. Push notification through OneSignal on a new TLS connection
if (uploaded) {
  WiFiClientSecure pushClient;  pushClient.setInsecure();
  HTTPClient pushHttp;
  pushHttp.begin(pushClient, "https://api.onesignal.com/notifications?c=push");
```

```
  pushHttp.addHeader("Content-Type", "application/json");
  pushHttp.addHeader("Authorization", "Key " + ONESIGNAL_REST_KEY);   // secret
  String payload = "{\"app_id\":\"" + ONESIGNAL_APP_ID + "\","
    "\"target_channel\":\"push\","
    "\"included_segments\":[\"Subscribed Users\"],"
    "\"headings\":{\"en\":\"Intrusion Detected!\"},"
    "\"contents\":{\"en\":\"Stealth movement detected. Tap to view the capture.\"},"
    "\"data\":{\"image_url\":\"" + imageUrl + "\"}}";
  int pc = pushHttp.POST(payload);
  pushHttp.end();
}

// 7. Wi-Fi off so that ADC2 (microphone) works again
WiFi.disconnect(true);  WiFi.mode(WIFI_OFF);
```

Security issues first: the OneSignal REST API key is able to send push notifications to all subscribed users to the application. It is essential to keep this key away from the place where it can be found, including any repository and particularly the mobile application itself. In a real-world scenario, the key would be stored on a small back-end server and not on the client side.

Wi-Fi credentials presented above are just for placeholder purposes.Second thing is that the setInsecure() call disables server certificate validation. It is okay to use it in a prototype with private Wi-Fi hotspot, but in reality the corresponding root certificates should be stored and server certificate verified on each secure connection.

7.7 Mobile application

The mobile application is built with Flutter. It uses the onesignal_flutter package to register the phone for push notifications and cached_network_image to show the captured photograph. The app is initialised with the OneSignal App ID only, which is not a secret, and the REST API key stays in the firmware. On first launch the app asks for permission to show notifications, and once permission is given the phone appears as a subscribed device in OneSignal. Since OneSignal delivers notifications through the platform's push service, alerts also arrive when the app is in the background.

I kept the interface simple: a title bar and a framed image of the latest capture. Before the first alert, the screen shows the message “No intrusion captured yet”. Two listeners from the OneSignal SDK update the image. The notification click listener runs when the user taps an alert, and the foreground listener handles notifications that arrive while the app is open. Both read the image_url field from the notification's additional data and use it to display the photograph.

Cloudinary gives every unsigned upload a new, randomly generated file name, so the image URL has to travel inside the notification and cannot be hard-coded in the app. The listing below shows the main part of the implementation.

```
void _updateFrom(Map<String, dynamic>? data) {
  final url = data?['image_url'];
  if (url is String && url.isNotEmpty && mounted) {
    setState(() => imageUrl = url);          // show the latest capture
  }
```

```
}

@override
void initState() {
  super.initState();
  OneSignal.Notifications.addClickListener((event) {
    _updateFrom(event.notification.additionalData);      // user taps the alert
  });
  OneSignal.Notifications.addForegroundWillDisplayListener((event) {
    _updateFrom(event.notification.additionalData);      // app is open
  });
}
```

7.8 Problems met and their solutions

Table 7.3 lists the problems I met while putting the parts together, what caused each one, and how I solved it. Several of them showed up only when the camera, Wi-Fi and microphone were used together, so they could not be found by testing each part alone.

Table 7.3: Problems met during implementation and their solutions

Problem	Cause	Solution
ADC reading stuck at 4095	60 dB default gain saturated the signal	GAIN pin connected to 3.3 V (40 dB)
Boot failure “invalid header 0xffffffff”	Microphone wire on strapping pin GPIO12	Output moved to GPIO14
Resets during camera and Wi-Fi use	Brown-out from current peaks	Separate 5 V supply
Upload failed with negative error code	Not enough internal RAM for TLS with the camera active	Copy frame to PSRAM and de-initialise the camera before upload
Microphone unreadable while uploading	ADC2 is unusable when Wi-Fi is on	Wi-Fi only on during events; off during listening
Garbled serial output	Baud-rate mismatch (9600 vs 115200)	Same rate on sketch and monitor
Push request failed with HTTP 401	Key used in the request was not a valid REST API key	Complete REST API key copied from the OneSignal settings
Push accepted but “All included players are not subscribed”	Application was started with a wrong identifier, so the phone never registered	Application initialised with the App ID; phone shows as subscribed
Push could not reuse the upload connection	The Cloudinary TLS connection was still open	Separate scope and new TLS client for the push request

RESULT DISCUSSION AND ANALYSIS

This chapter presents the results of the tests planned in Section 4.9. The data come from Edge Impulse Studio, the serial monitor and serial plotter of the prototype, and the logs of the alert system.

8.1 Hardware and signal verification

I started by verifying the sensing chain. I recorded the raw ADC readings of the MAX9814 with the serial plotter in two conditions, a normal room with no movement and a stealth intrusion, and Figure 8.1 shows both plots. In the normal room the signal stays roughly between 1000 and 1800, centred near the mid-scale value of about 1400. During the intrusion the variations are much stronger, with peaks of about 2160 and several drops into the 0–500 range.

Both readings stay within the 12-bit ADC range of 0 to 4095. With the default 60 dB gain the readings tended to saturate at 4095, whereas with the 40 dB setting the signal no longer sticks to the upper limit, which supports the choice of that setting. The two plots differ in amplitude and in how sharp the fluctuations are, and these are some of the patterns the Mel-based classifier has to separate. I tested the camera separately, first with the standard camera web-server example and then through the capture routine of the final firmware.

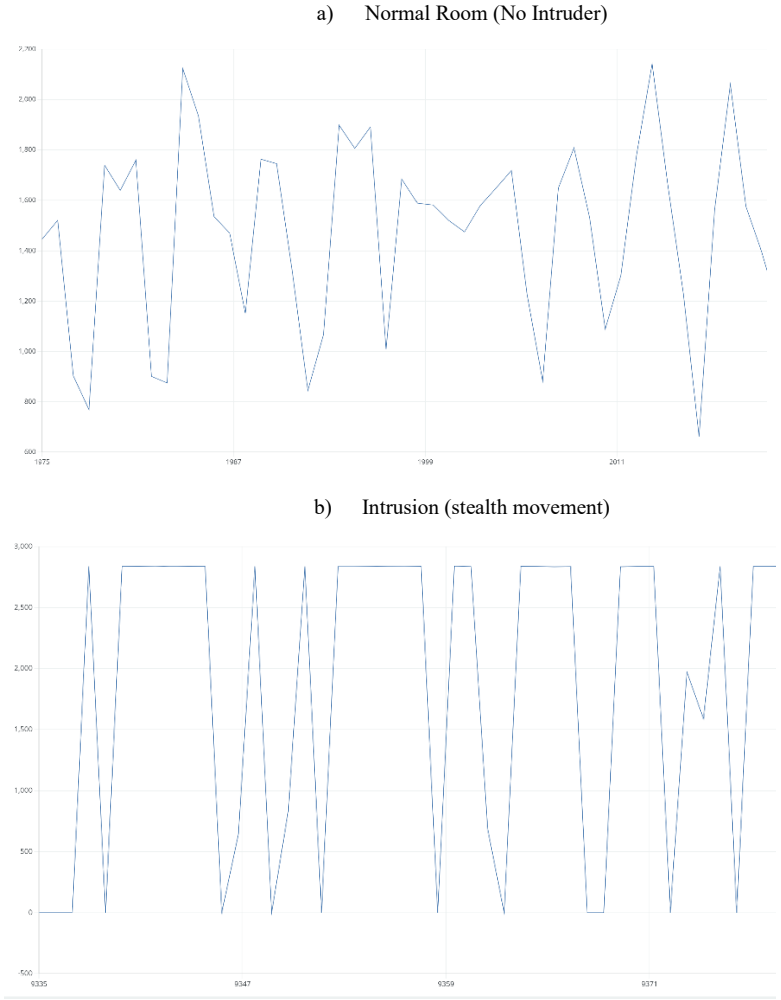


Figure 8.1: Raw microphone signal in the serial plotter: (a) normal room, (b) intrusion

8.2 Dataset summary

Table 8.1 summarises the dataset used in this project, giving the duration of the recordings, the number of persons and the number of sessions for each class.

Table 8.1: Summary of the collected dataset

Class	Duration	Persons	Sessions	Used for
stealth_movement	4 min 30 sec	4	10	Training / validation
normal_movement	3 min 36 sec	4	5	Training / validation

The dataset combines my own recordings, made with the setup of Figure 4.2, and public clips (Section 4.4). About 6 min 30 s of the total was recorded by me, and about 30 s each comes from the YouTube fan recording [31], the Freesound tiptoe clip [32] and the Pixabay clip [33].

8.3 Classifier performance

Figure 8.2 shows the results from Edge Impulse Studio. On the validation set the quantised int8 model reaches 93.8% accuracy with a loss of 0.14. On the test data the unoptimised float32 model reaches 90.09% accuracy and an area under the ROC curve (AUC) of 0.91, and its weighted average precision, recall and F1 score are all 0.91. Table 8.2 gives the confusion matrix for the test data, and Table 8.3 lists the per-class metrics reported by Edge Impulse Studio for both sets.

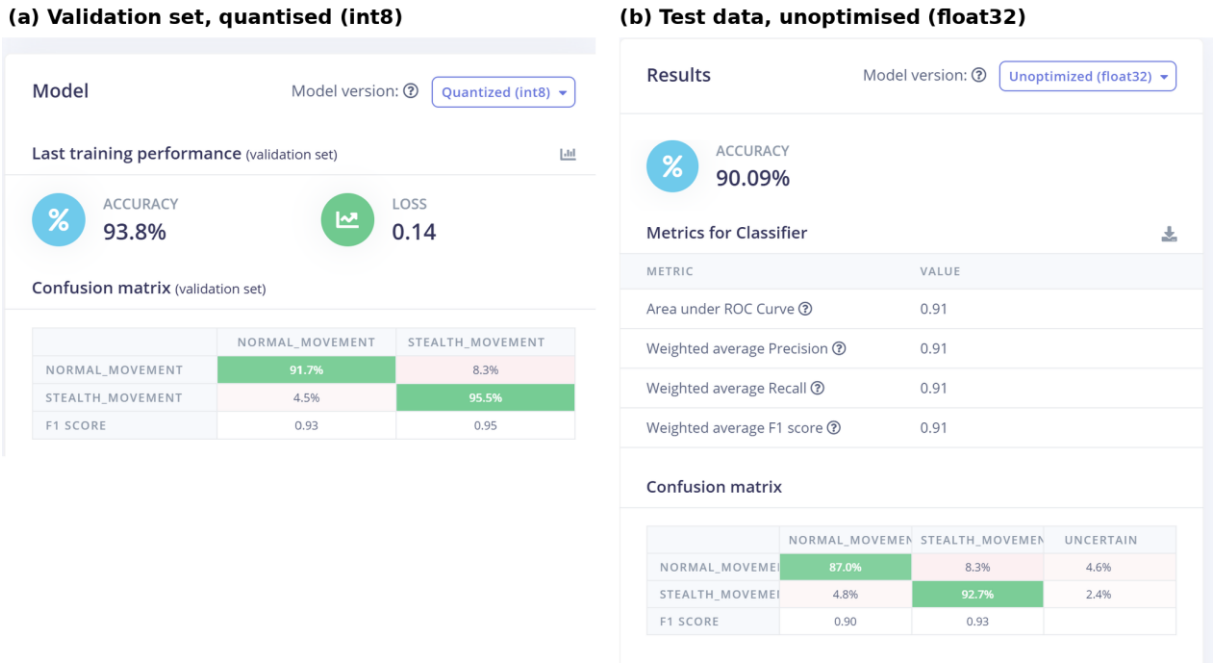


Figure 8.2: Results in Edge Impulse Studio: (a) validation set, quantised (int8) model; (b) test data, unoptimised (float32) model

Table 8.2: Confusion matrix on the test data (rows: true class, columns: predicted class)

True \ Predicted	normal_movement	stealth_movement	Uncertain
normal_movement	87.0%	8.3%	4.6%
stealth_movement	4.8%	92.7%	2.4%

Table 8.3: Per-class results and overall metrics

Quantity	Value
Accuracy	93.8% (validation); 90.09% (test)
Correct classification of stealth_movement windows	95.5% (validation); 92.7% (test)
Correct classification of normal_movement windows	91.7% (validation); 87.0% (test)
F1 score, normal_movement / stealth_movement	0.93 / 0.95 (validation); 0.90 / 0.93 (test)
Area under ROC curve	0.91 (test)
Weighted precision / recall / F1	0.91 / 0.91 / 0.91 (test)

As we see from the confusion matrix, there is a relatively small amount of stealth windows that are not detected in validation (4.5%) and test data (4.8%). On the other hand, there is also a relatively large number of normal windows which are detected as being stealthy (8.3% in both cases).

This kind of bias in detection can be considered to be a good thing for a security system if the goal is to detect any stealthy actions without making mistakes and the proposed algorithm decreases false positive alerts even more (see Section 8.4).

Uncertain windows are not considered to be correct for the test dataset, so this leads to a decrease in the accuracy from 93.8% in validation to 90.09%.

These two accuracies are not comparable since the first one refers to the deployed int8 model and the second one is for unoptimised float32 model and test sessions were not included in training dataset.

Figure 8.3 below shows the misclassified validation windows within Edge Impulse Studio. Here, all classification errors cluster around the bottom of the stealth window distribution area: when there is a high degree of noise or normality in normal activity, it becomes difficult to distinguish what is really going on. By implementing the confirmation rule and gathering additional data, I hope to eliminate many of these issues.



Figure 8.3: Data explorer (correctly classified normal windows hidden) and on-device performance estimate in Edge Impulse Studio

8.4 Effect of the confirmation rule

The serial log of the device shows the confirmation rule at work. Table 8.4 lists eight consecutive classification results with the hit counter, following the rule in Section 4.7. The high score in window 1 is an isolated hit, and the next four windows fall below the 0.70 threshold, so the counter resets and no alert is sent. An alert comes only after three consecutive windows exceed the threshold (windows 6–8). The scores are multiples of 1/256 (for example 0.996094 = 255/256), which matches the output resolution of an int8 quantised model and confirms that the device is running the quantised version.

Table 8.4: Stealth scores from the serial log and the resulting hit counter

Window	Stealth score	Above 0.70?	Hit counter	Action
1	0.996	Yes	1	None
2	0.051	No	0	None
3	0.188	No	0	None
4	0.188	No	0	None
5	0.012	No	0	None
6	0.996	Yes	1	None
7	0.996	Yes	2	None
8	0.996	Yes	3	ALERT

Without the confirmation rule, the single high score in window 1 would have switched on the camera, Wi-Fi and notification system. In the idle test, where the device ran for 30 minutes in a room without stealth movement, 6 false triggers were recorded, and the stealth-movement test gave 19 correct detections. These are counts from a small test and not a measured detection rate. A longer idle test with false triggers counted per hour is left for future work.

8.5 Resource usage of the deployed model

Memory use, flash requirements and latency are the main figures for judging whether a model can run on a resource-constrained device [26]. Table 8.5 gives the measurements reported by Edge Impulse Studio for the EON-compiled, RAM-optimised model (also visible in Figure 8.3), together with the free heap printed by the firmware before the network requests.

Table 8.5: Resource usage on the ESP32-WROVER

Quantity	Value
Inference time per window (Studio estimate)	1 ms
Peak RAM of the model (Studio estimate)	10.8 kB
Flash usage of the model (Studio estimate)	40.0 kB
Free heap before the Cloudinary upload (printed by the firmware)	about 199,000 bytes
Free heap before the push request (printed by the firmware)	about 198,000 bytes

The classifier takes only a small part of the module's memory, and its inference time of about 1 ms leaves most of the time between windows free for sampling and the confirmation logic. The free heap stays almost the same between the image upload and the push request, which tells me that the separate scope for the Cloudinary connection does its job. The Studio estimates are for the quantised int8 model compiled with the EON compiler and deployed on the device.

8.6 Upload and notification performance

During integration testing I logged five complete alert events, summarised in Table 8.6. All five image uploads to Cloudinary returned HTTP 200, with VGA photographs of roughly 12 kB to 53 kB. The first four notification attempts, however, failed or never reached a device because of configuration errors.

Two causes were behind this. Events 1 and 2 returned HTTP 401 because the key I used was not a valid REST API key, and events 3 and 4 were accepted by OneSignal but reached no device, because the app had been started with the wrong identifier and the phone was never registered. After I corrected both, the fifth event completed the whole alert process: OneSignal accepted the notification and returned a notification ID, and the alert arrived on the phone.

Table 8.6: Logged alert events during integration of the alert path

Event	Photo size (bytes)	Cloudinary reply	OneSignal reply
1	52,933	HTTP 200	HTTP 401, invalid key
2	16,240	HTTP 200	HTTP 401, invalid key
3	15,172	HTTP 200	HTTP 200, no subscribed device
4	15,267	HTTP 200	HTTP 200, no subscribed device
5	11,922	HTTP 200	HTTP 200, notification id returned

8.7 Comparative analysis

A numerical comparison with the papers in Chapter 2 is not possible, because they differ in sensors, data and metrics. Table 8.7 therefore compares the approaches by design properties, taken from the literature review and from this prototype.

Table 8.7: Comparison of the proposed system with other approaches

Approach	Trigger signal	Judges stealth?	On-device ML	Photo on event
PIR-triggered ESP32-CAM systems [2], [4]	Heat/motion	No	No	Yes
Seismic personnel detection [3], [14]	Ground vibration	No	Not stated	No
Acoustic footstep recognition [9], [10]	Footstep sound	No (counting/identity)	Not stated	No
Proposed system	Learned sound pattern	Yes	Yes (int8 1D CNN)	Yes, with push alert

The comparison shows what this system adds. Earlier approaches either detect movement without recognising quiet movement in particular, or do not report running on a microcontroller. This low-cost device combines learned quiet-movement classification, on-device inference in about 1 ms with 10.8 kB of RAM, and a cloud alert with a photograph. Because it relies on sound alone, the current prototype remains limited when a person makes almost no noise.

8.8 Discussion and limitations

These results should be read with the limits of my testing in mind. The validation accuracy of 93.8% and test accuracy of 90.09% were measured on the recorded dataset and not in real-world conditions, and so far I have shown field behaviour only through serial logs of individual events.

I recorded with one microphone in a limited number of rooms. Some background audio came from public sources recorded with different microphones, so the model may have picked up microphone-specific characteristics. Its accuracy on audio from the MAX9814 alone could therefore differ, and I cannot promise the same performance in a new room.

The model has only two classes, so quiet background sounds must be assigned to one of them, and the overlap in Figure 8.3 shows that some normal and stealth windows are hard to tell apart. A person moving extremely slowly, or using masking noise, could also get past a sound-based detector, which is why I want to add a vibration sensor in the next phase. Photographs taken at night depend on the room lighting, because the camera cannot produce clear images in complete darkness. Finally, the alert path depends on Wi-Fi and external services. If the network is unavailable, the firmware stops retrying, switches off the radio and goes back to listening, and that event is missed.

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

In this project, I developed an inexpensive detector of stealthy activity based on sound. The ESP32-WROVER board processes sound through MAX9814 microphone sampled at 8 kHz and classifies the sound using quantised Mel-filterbank neural network on the device. An alarm will be triggered only if confidence exceeds 0.70 during three successive intervals, with a 60 seconds cooldown period. Then the Wi-Fi network and OV2640 camera are turned on, the picture is uploaded to Cloudinary and a OneSignal notification is sent to the Flutter application. In case of normal listening, the Wi-Fi and the camera are turned off, which makes the battery-powered operation feasible in the future.

The model had 93.8% accuracy of validation and 90.09% accuracy of testing. 95.5% of the stealth intervals were detected by the model in the validation dataset. The model consumes 10.8 kB of RAM and 40.0 kB of flash memory and inference time equals 1 ms. The prototype performed the whole alerting process and the picture was sent together with the notification to the phone (Chapter 8).

The exercise has also offered me some useful insights. The gain of the microphone has to be tuned to ensure the ADC does not go into saturation mode, ADC2 pins cannot be activated when the Wi-Fi module is active, the camera needs to be de-initialised prior to a TLS connection for freeing up memory, and independent TLS connections have to be established for independent network transactions. What is currently lacking is an extended observation period for false trigger detection and further testing of the alerting mechanism (Chapter 8).

9.2 Future scope

- Improvement of Sensors – The inclusion of floor vibration sensors to correlate vibration and sound energy will also increase reliability against masking noise.
- Automatic Calibration – Adding a trigger (like pressing a button) to record the environment state of the room in order to calibrate the model and threshold to accommodate parameters like floor surface and noise level.
- Bigger Data Set and Model Comparison – Training the model using more individuals, footwear, rooms and distances; adding a separate class for noise; comparing other features like MFCC with MFE; as well as testing out other lightweight convolutional network models.
- Battery Operation – Adding a rechargeable battery and charging unit to increase portability. Deep sleep mode will be implemented where possible to test out the battery duration.
- Enhanced Security of Alert Notifications – Instead of using setInsecure(), add certificate verification and protect the key in the backend. Adding support for multiple recipients and storing captured images within the application is useful too.

- Night-Time Detection – Adding an infrared illuminator (or a simple IR-sensitive camera) could help detect people at night. Allowing a short video instead of a single photo could also be considered.

REFERENCES

[1] J. Fraden, *Handbook of Modern Sensors: Physics, Designs, and Applications*, 5th ed. Cham, Switzerland: Springer, 2016.

[2] K. Okokpujie, I. P. Okokpujie, F. T. Young, and R. E. Subair, "Development of an affordable real-time IoT-based surveillance system using ESP32 and TWILIO API," *Int. J. Saf. Secur. Eng.*, vol. 13, no. 6, pp. 1069-1075, Dec. 2023, doi: 10.18280/ijsse.130609.

[3] X. Jin, S. Gupta, A. Ray, and T. Damarla, "Multimodal sensor fusion for personnel detection," in *Proc. 14th Int. Conf. Inf. Fusion (FUSION)*, Chicago, IL, USA, Jul. 5-8, 2011.

[4] F. Humam and M. A. Triawan, "Sistem keamanan ruangan menggunakan ESP32CAM dan sensor gerak berbasis IoT," *Infotek: J. Inform. Teknol.*, vol. 7, no. 2, pp. 575-584, 2024, doi: 10.29408/jit.v7i2.26109.

[5] S. S. Saha, S. S. Sandha, and M. Srivastava, "Machine learning for microcontroller-class hardware: A review," *IEEE Sensors J.*, vol. 22, no. 22, pp. 21362-21390, 2022, doi: 10.1109/JSEN.2022.3210773.

[6] S. Garai and S. Samui, "Advances in small-footprint keyword spotting: A comprehensive review of efficient models and algorithms," 2025, arXiv:2506.11169.

[7] Y. Qu, "Acoustic detection and classification of human postures," Ph.D. dissertation, Dept. Elect. Eng. Electron., Univ. Liverpool, Liverpool, U.K., 2024.

[8] H. L. Hawk, J. G. Hawley, J. M. Portlock, and J. E. Scheibner, "Seismic intrusion detector system," U.S. Patent 3 984 803, Oct. 5, 1976.

[9] Y. Qu, L. Shi, X. Wang, and Z. Wang, "Indoor multi-person detection and recognition through footsteps: A deep learning approach with acoustic signal analysis," *IEEE Sensors J.*, vol. 24, no. 12, pp. 19482-19496, 2024, doi: 10.1109/JSEN.2024.3394212.

[10] L. Huang and C. Wang, "Unobtrusive pedestrian identification by leveraging footstep sounds with replay resistance," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 5, no. 4, pp. 1-19, 2021, doi: 10.1145/3494963.

[11] M. Huzaifah, "Comparison of time-frequency representations for environmental sound classification using convolutional neural networks," 2017, arXiv:1706.07156.

[12] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning on TinyML systems," in *Proc. Mach. Learn. Syst. (MLSys)*, 2021, arXiv:2010.08678.

[13] Espressif Systems, "ESP32 series datasheet," version 5.3. [Online]. Available: https://www.espressif.com.

[14] P. S. Huang, T. Damarla, and M. Hasegawa-Johnson, "Multi-sensory features for personnel detection at border crossings," in *Proc. 14th Int. Conf. Inf. Fusion (FUSION)*, 2011.

[15] A. F. R. Nogueira, H. S. Oliveira, J. J. M. Machado, and J. M. R. S. Tavares, "Sound classification and processing of urban environments: A systematic literature review," *Sensors*, vol. 22, no. 22, Art. no. 8608, Nov. 2022, doi: 10.3390/s22228608.

[16] Z. Zhang, S. Xu, S. Cao, and S. Zhang, "Deep convolutional neural network with mixup for environmental sound classification," 2018, arXiv:1808.08405.

[17] S. Hymel *et al.*, "Edge Impulse: An MLOps platform for tiny machine learning," in *Proc. 6th Conf. Mach. Learn. Syst. (MLSys)*, Miami Beach, FL, USA, 2023, arXiv:2212.03332.

[18] V. Viswanatha *et al.*, "Implementation of tiny machine learning models on Arduino 33 BLE for gesture and speech recognition," 2022, arXiv:2207.12866.

[19] A. Barovic and A. Moin, "TinyML for speech recognition," 2025, arXiv:2504.16213.

[20] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 28, no. 4, pp. 357-366, Aug. 1980.

[21] Cloudinary, "Image upload API reference." [Online]. Available: https://cloudinary.com/documentation.

[22] OneSignal, "OneSignal documentation: Create notification API." [Online]. Available: https://documentation.onesignal.com.

[23] Espressif Systems, "ESP-IDF programming guide: FreeRTOS." [Online]. Available: https://docs.espressif.com/projects/esp-idf.

[24] Espressif Systems, "esp32-camera: ESP32 camera driver," GitHub repository. [Online]. Available: https://github.com/espressif/esp32-camera.

[25] Maxim Integrated (now Analog Devices), "MAX9814: Microphone amplifier with AGC and low-noise microphone bias," datasheet. [Online]. Available: https://www.analog.com/en/products/max9814.html.

[26] C. R. Banbury *et al.*, "Benchmarking TinyML systems: Challenges and direction," 2020, arXiv:2003.04821.

[27] D. Stowell, D. Giannoulis, E. Benetos, M. Lagrange, and M. D. Plumbley, "Detection and classification of acoustic scenes and events," *IEEE Trans. Multimedia*, vol. 17, no. 10, pp. 1733-1746, Oct. 2015.

[28] K. J. Piczak, "ESC: Dataset for environmental sound classification," in *Proc. 23rd ACM Int. Conf. Multimedia*, 2015, pp. 1015-1018.

[29] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018, arXiv:1804.03209.

[30] Edge Impulse, "Edge Impulse documentation." [Online]. Available: https://docs.edgeimpulse.com.

[31] Randall's Relaxation, "10 Hours of CEILING FAN NOISE for BEDROOM FAN SOUNDS FOR SLEEPING," YouTube, 2017. [Online]. Available: https://www.youtube.com/watch?v=5E67XUpbiIE&t=6s. Accessed: Oct. 2026.

[32] taure, "tiptoe.wav," Freesound, Jun. 10, 2021. [Online]. Available: https://freesound.org/people/taure/sounds/575321/. Accessed: Oct. 2026.

[33] freesound_community, "Spy," Pixabay. Pixabay Content License. Accessed: Oct. 2026.

[34] S. Hershey *et al.*, "CNN architectures for large-scale audio classification," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2017, pp. 131-135.

[35] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704-2713.

Tanishq Yadav

Final Tanishq

plagiarism and AI-generated content similarity reports

5th Sem. Minor Project

Indian Institute of Information Technology BP

Document Details

Submission ID
trn:oid:::1:3697193275

Submission Date
Nov 1, 2026, 10:31 AM GMT+5:30

Download Date
Nov 1, 2026, 10:41 AM GMT+5:30

File Name
Tanishq_minorProject.pdf

File Size
1.2 MB

49 Pages

15,810 Words

83,680 Characters

35% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

68 AI-generated 35%
Likely generated or likely generated and revised by AI.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Tanishq Yadav

Final Tanishq

plagiarism and AI-generated content similarity reports

5th Sem. Minor Project

Indian Institute of Information Technology BP

Document Details

Submission ID
trn:oid:::1:3697193275

Submission Date
Nov 1, 2026, 10:31 AM GMT+5:30

Download Date
Nov 1, 2026, 10:40 AM GMT+5:30

File Name
Tanishq_minorProject.pdf

File Size
1.2 MB

49 Pages

15,810 Words

83,680 Characters

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

97 Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks

1 Missing Quotations 0%
Matches that are still very similar to source material

6 Missing Citation 1%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
7% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.